

# SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

## ASSESSMENT TOOL 3 – Comparative Analysis

|                  |                                                                              |               |                                          |
|------------------|------------------------------------------------------------------------------|---------------|------------------------------------------|
| Course Code:     | CSA06                                                                        | Course Title: | Design and Analysis of Algorithms        |
| CO Assessed:     | CO4: Articulation (BL4, BL5)                                                 | Assessment:   | Assessment Tool 3 – Comparative Analysis |
| Weightage:       | 10% of CIA                                                                   | Total Marks:  | 100                                      |
| CO4 Statement:   | Solve optimization problems using dynamic programming and greedy strategies. |               |                                          |
| Student Name:    | LOKESH S                                                                     | Reg. No.:     | 192572151                                |
| Section / Batch: | CSE (AI) /2 <sup>ND</sup> YEAR                                               | Date:         | 14/08/2026                               |

### Instructions to Students

- Answer the following question. Total Marks: 100.
- Analyze the given questions thoroughly before answering.
- Compare multiple aspects such as methods, results, advantages, and limitations clearly.
- Critically evaluate the strengths, weaknesses, and implications with proper justification.
- Present meaningful insights, interpretations, and well-supported conclusions from the comparisons.

| Question Type        | Focus Area                                                                                   | Questions |
|----------------------|----------------------------------------------------------------------------------------------|-----------|
| Comparative Analysis | Comparative analysis of Dynamic Programming and Greedy algorithms for optimization problems. | Q1        |

### SECTION A – Comparative Analysis

#### Code of Conduct

I LOKESH S (192572151) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: LOKESH S

## RUBRICS FOR CALCULATION

| Comparative Analysis            |           |                                                                                               |                                                                  |                                         |                                               |
|---------------------------------|-----------|-----------------------------------------------------------------------------------------------|------------------------------------------------------------------|-----------------------------------------|-----------------------------------------------|
| Criteria                        | Weightage | Level 4 (Exemplary)                                                                           | Level 3 (Proficient)                                             | Level 2 (Developing)                    | Level 1 (Beginning)                           |
| Selection of Studies/Parameters | 20%       | Selects highly relevant and diverse studies with well-defined comparison parameters           | Selects relevant studies with minor gaps in parameter definition | Limited or partially relevant selection | Poor or inappropriate selection of studies    |
| Comparison & Differentiation    | 25%       | Clearly compares multiple aspects (methods, results, limitations) with strong differentiation | Good comparison with minor gaps                                  | Basic comparison with limited depth     | No clear comparison; mostly descriptive       |
| Critical Evaluation             | 25%       | Critically evaluates strengths, weaknesses, and implications with strong justification        | Provides evaluation with some justification                      | Limited evaluation; lacks depth         | No critical evaluation provided               |
| Synthesis & Insight Generation  | 20%       | Generates meaningful insights and conclusions from comparisons                                | Some insights derived from comparison                            | Limited or unclear insights             | No insights generated                         |
| Clarity                         | 10%       | Information is clearly presented (tables/figures) with logical structure                      | Generally clear with minor issues                                | Presentation lacks clarity or structure | Poor presentation and difficult to understand |

### Marks Evaluation:

| Criteria                        | Wt. | Level 4 – Exemplary | Level 3 – Proficient | Level 2 – Developing | Level 1 – Beginning |
|---------------------------------|-----|---------------------|----------------------|----------------------|---------------------|
| Selection of Studies/Parameters | 20% |                     |                      |                      |                     |
| Comparison & Differentiation    | 25% |                     |                      |                      |                     |
| Critical Evaluation             | 25% |                     |                      |                      |                     |
| Synthesis & Insight Generation  | 20% |                     |                      |                      |                     |
| Clarity                         | 10% |                     |                      |                      |                     |
| <b>Total Marks (100):</b>       |     |                     |                      |                      |                     |

| Faculty In-charge | Course Coordinator | Head of Department |
|-------------------|--------------------|--------------------|
| Name:             | Name:              | Name:              |
| Signature: _____  | Signature: _____   | Signature: _____   |
| Date: _____       | Date: _____        | Date: _____        |

## 1. Knapsack Comparison (DP vs Greedy)

Items: weights [10, 20, 30], values [60, 100, 120], capacity = 50.

### 0/1 Knapsack (Dynamic Programming)

Only whole items may be taken. Evaluating all feasible combinations under capacity 50:

| Combination   | Weight          | Value |
|---------------|-----------------|-------|
| Item1 + Item2 | 30              | 160   |
| Item1 + Item3 | 40              | 180   |
| Item2 + Item3 | 50              | 220   |
| All three     | 60 (infeasible) | —     |

Optimal 0/1 solution: Items 2 and 3 (weight 50, value 220).

### Fractional Knapsack (Greedy)

Greedy sorts by value/weight ratio: Item1 = 6.0, Item2 = 5.0, Item3 = 4.0.

- Take Item1 fully: weight 10, value 60, remaining capacity 40
- Take Item2 fully: weight 20, value 100, remaining capacity 20
- Take 20/30 (66.7%) of Item3: value 80

Total value = 60 + 100 + 80 = 240 (weight exactly 50).

### Comparison

| Aspect          | 0/1 Knapsack (DP)                 | Fractional Knapsack (Greedy)               |
|-----------------|-----------------------------------|--------------------------------------------|
| Optimal value   | 220                               | 240                                        |
| Items used      | Item2 + Item3 (whole)             | Item1 + Item2 + 66.7% of Item3             |
| Time complexity | $O(nW)$                           | $O(n \log n)$                              |
| Optimality      | Optimal for whole-item constraint | Optimal only because fractions are allowed |

### Analysis and Insights

- Greedy by ratio is only optimal when items can be split (fractional knapsack has the greedy-choice and optimal-substructure properties simultaneously).
- If greedy is applied to the 0/1 version by taking whole items in ratio order, it yields Item1 + Item2 = 160, worse than the DP optimum of 220 — greedy commits early and cannot backtrack.

- DP is necessary whenever choices are indivisible and a later decision can invalidate an earlier 'locally best' choice.
- Strength of greedy: near-linear time, simple to implement. Limitation: no correctness guarantee under indivisibility.
- Strength of DP: guaranteed optimal for 0/1 case. Limitation: pseudo-polynomial time/space (depends on capacity  $W$ ), impractical for very large  $W$ .

## 2. Shortest Path Comparison (Dijkstra vs Bellman-Ford)

Graph (directed), vertices A, B, C, D; edges:  $A \rightarrow B = 1$ ,  $A \rightarrow C = 4$ ,  $B \rightarrow C = -3$ ,  $B \rightarrow D = 2$ ,  $C \rightarrow D = 3$ .

### Bellman-Ford from A

Relax all edges repeatedly ( $|V|-1 = 3$  passes):

- $\text{dist}(A) = 0$
- $\text{dist}(B) = 1$  (via  $A \rightarrow B$ )
- $\text{dist}(C) = \min(A \rightarrow C = 4, B \rightarrow C = 1 + (-3) = -2) = -2$
- $\text{dist}(D) = \min(B \rightarrow D = 1 + 2 = 3, C \rightarrow D = -2 + 3 = 1) = 1$

Bellman-Ford result:  $A=0$ ,  $B=1$ ,  $C=-2$ ,  $D=1$  — correct, and it also detects that no negative cycle exists.

### Dijkstra from A

Dijkstra greedily finalizes the closest unvisited vertex and never revisits it:

- Finalize  $A = 0$ .
- Finalize  $B = 1$  (smallest tentative distance).
- Finalize  $C = 4$  (via  $A \rightarrow C$ ), since  $B \rightarrow C$  has not been considered —  $C$  was already 'settled' relative to what was known, or if  $C$  is updated to  $1 + (-3) = -2$  after  $B$  is finalized, standard Dijkstra still will not re-open  $B$  or earlier finalized nodes, and with a proper implementation the negative edge can even produce  $\text{dist}(C) = -2$  but downstream nodes that were finalized before this update remain wrong.
- In general form: once  $B \rightarrow C = -3$  is processed after  $C$ ,  $D$  are finalized using stale values, Dijkstra reports incorrect shortest paths, since it assumes distances only increase as more edges are traversed.

## Comparison

| Aspect                   | Dijkstra                          | Bellman-Ford                 |
|--------------------------|-----------------------------------|------------------------------|
| Handles negative weights | No (unsafe/incorrect)             | Yes                          |
| Negative-cycle detection | No                                | Yes                          |
| Time complexity          | $O((V+E) \log V)$ with a min-heap | $O(V \cdot E)$               |
| Approach                 | Greedy, finalizes closest vertex  | DP-style repeated relaxation |

## Analysis and Insights

- Dijkstra's correctness proof relies on the invariant that once a vertex is finalized, no shorter path to it can appear later — negative edges break this invariant.
- Bellman-Ford trades speed for correctness under negative weights, and additionally detects negative cycles, which Dijkstra cannot do at all.
- Performance: for large graphs with only non-negative weights, Dijkstra is markedly faster and should be preferred; Bellman-Ford is reserved for graphs where negative edges (e.g., cost reductions, arbitrage-style graphs) are possible.

### 3. MST Comparison (Prim vs Kruskal)

Graph: vertices A, B, C, D, E; edges A-B=2, A-C=3, B-C=1, B-D=4, C-D=5, C-E=6, D-E=7.

#### Kruskal's Algorithm

Sort edges by weight: B-C(1), A-B(2), A-C(3), B-D(4), C-D(5), C-E(6), D-E(7).

- Add B-C (1) — no cycle.
- Add A-B (2) — no cycle.
- Skip A-C (3) — would form a cycle A-B-C.
- Add B-D (4) — no cycle.
- Skip C-D (5) — cycle.
- Add C-E (6) — no cycle. 4 edges reached for 5 vertices; stop.

Kruskal MST edges: B-C, A-B, B-D, C-E. Total weight = 13.

#### Prim's Algorithm (starting at A)

- Frontier from A: A-B(2), A-C(3) → pick A-B (2).
- Frontier: A-C(3), B-C(1), B-D(4) → pick B-C (1).
- Frontier: B-D(4), C-D(5), C-E(6) → pick B-D (4).
- Frontier: C-D(5, cycle), C-E(6), D-E(7) → pick C-E (6).

Prim MST edges: A-B, B-C, B-D, C-E. Total weight = 13 (same tree, edges selected in a different order).

#### Comparison

| Aspect          | Prim's Algorithm                              | Kruskal's Algorithm                                                     |
|-----------------|-----------------------------------------------|-------------------------------------------------------------------------|
| Growth strategy | Grows one connected tree from a start vertex  | Adds globally cheapest edge that avoids a cycle, across the whole graph |
| Data structure  | Min-heap / priority queue over frontier edges | Sorted edge list + Union-Find (disjoint set)                            |
| Time complexity | $O(E \log V)$ with a binary heap              | $O(E \log E)$                                                           |
| Best suited for | Dense graphs (adjacency matrix, many edges)   | Sparse graphs (edge list is small relative to $V^2$ )                   |

## 4. Coin Change Problem (Greedy vs DP)

Denominations: [1, 3, 4]. Amount: 6.

### Greedy Approach

- Pick largest coin  $\leq 6 \rightarrow 4$ , remainder 2
- Pick largest coin  $\leq 2 \rightarrow 1$ , remainder 1
- Pick largest coin  $\leq 1 \rightarrow 1$ , remainder 0

Greedy result:  $4 + 1 + 1 = 3$  coins.

### Dynamic Programming Approach

Minimum coins table for amounts 0..6 ( $\text{min}[0]=0$ ):

| Amount    | 0 | 1 | 2 | 3 | 4 | 5 | 6 |
|-----------|---|---|---|---|---|---|---|
| Min coins | 0 | 1 | 2 | 1 | 1 | 2 | 2 |

DP finds  $6 = 3 + 3$ , using only 2 coins — strictly better than greedy's 3 coins.

### Comparison

| Aspect                  | Greedy                                    | Dynamic Programming                                      |
|-------------------------|-------------------------------------------|----------------------------------------------------------|
| Coins used for amount 6 | $4 + 1 + 1 = 3$ coins                     | $3 + 3 = 2$ coins                                        |
| Correctness             | Fails for this coin set                   | Always optimal                                           |
| Time complexity         | $O(\text{amount} / \text{smallest coin})$ | $O(\text{amount} \times \text{number of denominations})$ |

### Analysis and Insights

- Greedy coin change is optimal only for 'canonical' coin systems (e.g., standard currency like 1, 5, 10, 25) where taking the largest denomination never blocks a better later combination.
- [1, 3, 4] is a non-canonical system: greedily taking 4 first strands a remainder of 2 that cannot be paid as cheaply as if 3+3 had been used instead.
- DP explores all sub-problems and is guaranteed correct for any denomination set, at the cost of higher time/space compared to greedy's near-constant per-step work.

## 5. Activity Selection (Greedy vs DP)

Start times: [1, 3, 0, 5, 8, 5]. Finish times: [2, 4, 6, 7, 9, 9].

| Activity | 1 | 2 | 3 | 4 | 5 | 6 |
|----------|---|---|---|---|---|---|
| Start    | 1 | 3 | 0 | 5 | 8 | 5 |
| Finish   | 2 | 4 | 6 | 7 | 9 | 9 |

### Greedy Approach (sort by finish time)

Sorted by finish time: A1(1,2), A2(3,4), A3(0,6), A4(5,7), A5(8,9), A6(5,9).

- Select A1 (1,2). Last finish = 2.
- A2 starts at  $3 \geq 2 \rightarrow$  select. Last finish = 4.
- A3 starts at  $0 < 4 \rightarrow$  skip.
- A4 starts at  $5 \geq 4 \rightarrow$  select. Last finish = 7.
- A5 starts at  $8 \geq 7 \rightarrow$  select. Last finish = 9.
- A6 starts at  $5 < 9 \rightarrow$  skip.

Greedy selects: A1, A2, A4, A5 — 4 activities.

### Dynamic Programming (Weighted/General Activity Selection)

Formulating this as an interval-scheduling DP (max activities ending at or before each point, then taking the best chain) also yields a maximum of 4 non-overlapping activities, confirming the same selection: A1, A2, A4, A5.

### Comparison

| Aspect          | Greedy                               | Dynamic Programming                   |
|-----------------|--------------------------------------|---------------------------------------|
| Result          | 4 activities: A1, A2, A4, A5         | 4 activities: A1, A2, A4, A5 (same)   |
| Time complexity | $O(n \log n)$ — dominated by sorting | $O(n^2)$ typical formulation          |
| Necessity       | Sufficient and optimal here          | Unnecessary overhead for this variant |

### Analysis and Insights

- Activity selection (unweighted, maximize count) satisfies the greedy-choice property: always picking the activity with the earliest finish time never eliminates an optimal solution — this is provable by an exchange argument.
- Because the greedy choice is provably safe, DP is not needed for this exact variant; DP becomes necessary only when activities carry weights/values (weighted interval scheduling), where greedy by finish time is no longer guaranteed optimal.



## 6. All-Pairs vs Single-Source Shortest Path (Floyd-Warshall vs Dijkstra)

Example network: vertices A, B, C, D with edges  $A \rightarrow B=3$ ,  $A \rightarrow C=8$ ,  $B \rightarrow D=1$ ,  $C \rightarrow D=2$ ,  $B \rightarrow C=4$  (all non-negative for a fair Dijkstra comparison).

### Dijkstra (single-source, from A)

- $\text{dist}(A) = 0$
- $\text{dist}(B) = 3$  ( $A \rightarrow B$ )
- $\text{dist}(D) = 4$  ( $A \rightarrow B \rightarrow D = 3+1$ )
- $\text{dist}(C) = \min(8, 3+4=7, 4+2=6) = 6$  ( $A \rightarrow B \rightarrow D \rightarrow C$ )

Dijkstra gives shortest paths from A only:  $A=0$ ,  $B=3$ ,  $C=6$ ,  $D=4$ .

### Floyd-Warshall (all pairs)

Running the triple-loop DP over all vertex pairs yields a complete distance matrix, including paths not rooted at A,

e.g.  $\text{dist}(B,C) = 3$  (via D:  $B \rightarrow D \rightarrow C = 1+2$ ),  $\text{dist}(C,B) = \infty$  or via any back-edges if present,  $\text{dist}(B,D) = 1$ ,  $\text{dist}(C,D) = 2$  — in effect the single-source table above is exactly the 'row for A' inside this larger matrix.

### Comparison

| Aspect             | Dijkstra                                    | Floyd-Warshall                           |
|--------------------|---------------------------------------------|------------------------------------------|
| Scope              | Single source to all vertices               | Every pair of vertices                   |
| Time complexity    | $O((V+E) \log V)$ per source                | $O(V^3)$ total                           |
| Cost for all-pairs | $O(V(V+E) \log V)$ if run from every vertex | $O(V^3)$ , often cheaper on dense graphs |
| Negative weights   | Not safe                                    | Safe (no negative cycles)                |

### Analysis and Insights

- For a single source query, Dijkstra is the clear choice — it avoids computing irrelevant pairwise distances.
- For dense graphs needing every pair's distance, Floyd-Warshall's  $O(V^3)$  is competitive with, or better than, running Dijkstra  $V$  times ( $O(V^3 \log V)$  for  $V \cdot (V+E)$  with dense  $E \approx V^2$ ).
- Floyd-Warshall also tolerates negative edges (not negative cycles), giving it broader applicability, at the cost of always paying the full  $O(V^3)$ , even if only one pair's distance is needed.

## 7. String Matching: LCS (DP) vs Greedy

Strings:  $X = \text{"ABCB DAB"}$  (length 7),  $Y = \text{"BDCAB"}$  (length 5).

### Longest Common Subsequence (Dynamic Programming)

Filling the standard LCS DP table ( $\text{dp}[i][j] = \text{LCS length of } X[1..i], Y[1..j]$ ) gives a final value of  $\text{dp}[7][5] = 4$ . One valid LCS obtained by backtracking: "B C A B" (characters at X-positions 2,3,6,7 and Y-positions 1,3,4,5).

### Greedy Matching Attempt

A naive greedy scan (match the next unmatched character of  $Y$  as soon as it is seen while scanning  $X$  left to right, never reconsidering a choice) proceeds as follows:

- Scanning  $X = \text{A,B,C,B,D,A,B}$  looking for  $Y = \text{B,D,C,A,B}$  in order.
- Need 'B' first: matches  $X[2]$ . Need 'D' next: matches  $X[5]$ . Need 'C' next:  $X$  has no C after position 5  $\rightarrow$  greedy gets stuck.
- Greedy subsequence found: "B D" — only length 2, and it cannot be extended further.

### Comparison

| Aspect          | Greedy Matching           | LCS (Dynamic Programming) |
|-----------------|---------------------------|---------------------------|
| Result          | "B D" — length 2          | "B C A B" — length 4      |
| Correctness     | Suboptimal / can dead-end | Guaranteed optimal        |
| Time complexity | $O(m+n)$ but unreliable   | $O(m \times n)$           |

### Analysis and Insights

- Greedy fails because an early local match (choosing 'D' right after 'B') forecloses better options later — LCS does not have a simple greedy-choice property, since the best match for a prefix depends on choices made over the whole remaining suffixes of both strings.
- LCS does have optimal substructure (the LCS of two strings is built from LCS's of their prefixes), which is exactly what DP exploits by considering both 'skip a character in  $X$ ' and 'skip a character in  $Y$ ' at every cell.

## 8. Travelling Salesman Problem (DP vs Greedy Heuristic)

4 cities (1,2,3,4) with symmetric cost matrix:

|   | 1  | 2  | 3  | 4  |
|---|----|----|----|----|
| 1 | 0  | 10 | 15 | 20 |
| 2 | 10 | 0  | 35 | 25 |
| 3 | 15 | 35 | 0  | 30 |
| 4 | 20 | 25 | 30 | 0  |

### Dynamic Programming (Held-Karp)

Evaluating all distinct tours starting/ending at city 1:

| Tour      | Cost               |
|-----------|--------------------|
| 1-2-3-4-1 | $10+35+30+20 = 95$ |
| 1-2-4-3-1 | $10+25+30+15 = 80$ |
| 1-3-2-4-1 | $15+35+25+20 = 95$ |
| 1-3-4-2-1 | $15+30+25+10 = 80$ |
| 1-4-2-3-1 | $20+25+35+15 = 95$ |
| 1-4-3-2-1 | $20+30+35+10 = 95$ |

Held-Karp DP optimum = 80, achieved by tour 1-2-4-3-1 (equivalently its reverse 1-3-4-2-1).

### Greedy Heuristic (Nearest Neighbor, from city 1)

- At 1: nearest unvisited is 2 (cost 10).
- At 2: nearest unvisited is 4 (cost 25, vs city 3 at 35).
- At 4: only 3 remains (cost 30).
- Return to 1 (cost 15).

Nearest-Neighbor tour: 1-2-4-3-1, total =  $10+25+30+15 = 80$  — matches the DP optimum in this particular instance, though nearest-neighbor is not guaranteed to do so in general.

### Comparison

| Aspect                  | DP (Held-Karp)               | Greedy (Nearest Neighbor)          |
|-------------------------|------------------------------|------------------------------------|
| Result on this instance | 80 (optimal)                 | 80 (matched optimum here)          |
| Guarantee               | Always optimal               | No optimality guarantee in general |
| Time complexity         | $O(n^2 \cdot 2^n)$           | $O(n^2)$                           |
| Scalability             | Infeasible beyond ~20 cities | Scales to thousands of cities      |

## Analysis and Insights

- Held-Karp's exponential state space ( $2^n$  subsets) makes exact DP impractical for large  $n$ , despite guaranteeing the true optimum.
- Nearest Neighbor happened to reach the optimal tour here because the instance is small and its cost structure is favorable; on larger or more adversarial instances, Nearest Neighbor is well known to produce tours substantially above optimal (no constant-factor guarantee in general, though bounded heuristics like Christofides exist for metric TSP).

## 9. Matrix Chain Multiplication (DP vs Greedy)

Matrices:  $A_1$  ( $10 \times 20$ ),  $A_2$  ( $20 \times 30$ ),  $A_3$  ( $30 \times 40$ ).

### Dynamic Programming

| Parenthesization | Scalar Multiplications                                                       |
|------------------|------------------------------------------------------------------------------|
| $(A_1 A_2) A_3$  | $(10 \times 20 \times 30) + (10 \times 30 \times 40) = 6000 + 12000 = 18000$ |
| $A_1 (A_2 A_3)$  | $(20 \times 30 \times 40) + (10 \times 20 \times 40) = 24000 + 8000 = 32000$ |

DP optimum:  $(A_1 A_2) A_3$ , with 18,000 scalar multiplications.

### Naive Greedy Approach

A greedy strategy that always multiplies the pair with the lowest immediate cost first evaluates:  $A_1 \times A_2$  costs 6000,  $A_2 \times A_3$  costs 24000 — so it also picks  $A_1 \times A_2$  first, coincidentally matching DP's choice for this 3-matrix chain.

For longer chains, this local 'cheapest immediate product' greedy strategy is known to fail: minimizing each individual step's cost ignores how the resulting matrix dimensions affect all subsequent multiplications, so a locally cheap merge can produce a much more expensive intermediate matrix shape for the next steps.

### Comparison

| Aspect                     | Greedy (local minimization)        | Dynamic Programming         |
|----------------------------|------------------------------------|-----------------------------|
| Result here (3 matrices)   | 18,000 (matches DP by coincidence) | 18,000 (guaranteed optimal) |
| Guarantee on longer chains | None — can be far from optimal     | Always optimal              |
| Time complexity            | $O(n)$ per greedy step             | $O(n^3)$                    |

## 10. Huffman Coding vs Fixed-Length Coding

Symbol frequencies: A:5, B:9, C:12, D:13, E:16, F:45 (total = 100).

### Building the Huffman Tree

- Merge A(5) + B(9) → node AB(14).
- Merge C(12) + D(13) → node CD(25).
- Merge AB(14) + E(16) → node ABE(30).
- Merge CD(25) + ABE(30) → node(55).
- Merge F(45) + node(55) → root(100).

### Resulting Huffman Codes

| Symbol | Frequency | Code | Code Length | Weighted Bits |
|--------|-----------|------|-------------|---------------|
| F      | 45        | 0    | 1           | 45            |
| C      | 12        | 100  | 3           | 36            |
| D      | 13        | 101  | 3           | 39            |
| E      | 16        | 111  | 3           | 48            |
| A      | 5         | 1100 | 4           | 20            |
| B      | 9         | 1101 | 4           | 36            |

Total Huffman-encoded size = 45+36+39+48+20+36 = 224 bits. Average code length = 224/100 = 2.24 bits/symbol.

### Fixed-Length Coding

6 distinct symbols require  $\text{ceil}(\log_2 6) = 3$  bits per symbol regardless of frequency.

Total fixed-length size = 100 symbols × 3 bits = 300 bits. Average code length = 3.0 bits/symbol.

### Comparison

| Aspect            | Huffman Coding               | Fixed-Length Coding |
|-------------------|------------------------------|---------------------|
| Total bits        | 224                          | 300                 |
| Avg. bits/symbol  | 2.24                         | 3.00                |
| Compression saved | 76 bits (≈25.3%)             | —                   |
| Code type         | Variable-length, prefix-free | Fixed-length        |

### Analysis and Insights

- Huffman coding is a greedy algorithm (always merge the two lowest-frequency nodes) that is provably optimal among prefix-free codes, because merging the two rarest symbols first never increases the final

weighted path length — a genuine greedy-choice property holds here, unlike several other cases in this report.

- Fixed-length coding is simple and allows  $O(1)$  random access to the  $n$ -th symbol, but wastes bits on frequent symbols (F, at 45% of occurrences, still costs 3 bits instead of the 1 bit Huffman assigns it).
- Limitation of Huffman: variable-length codes complicate random access and require transmitting/storing the code table; also assumes static, known frequencies (adaptive variants exist for streaming data).

